

HDX-ADM-100: DX Administration for Beginners

Study Guide (mapped to SAP Commerce / Hybris)

Contents

Course 1 — HDX-ADM-100: DX Administration for Beginners	1
THEORY GUIDE (mapped to SAP Commerce / Hybris)	1
1. The mental model: what is DX, administratively?	1
2. Practitioner Studio — your one console	2
3. Security model — the biggest conceptual jump from Hybris	2
4. DXClient — DX’s “deployment CLI”	3
5. SoFy / HCLSoftware Marketplace — the sandbox layer	4
Summary cheat-table	4
Course 1 — HDX-ADM-100: DX Administration for Beginners	6
PRACTICAL CHEAT SHEET	6
A. Deploy a sandbox (HDX-INTRO)	6
B. Practitioner Studio basics (HDX-ADM-100 Practitioner Studio Admin)	6
C. Security lab (HDX-ADM-100 DX Security)	7
D. DXClient install & configure (HDX-ADM-100 DXClient)	7
Quick command reference (grows in Course 2)	9

Course 1 — HDX-ADM-100: DX Administration for Beginners

THEORY GUIDE (mapped to SAP Commerce / Hybris)

Source docs: *HDX-INTRO Setup DX Solution Modules in SoFy · HDX-ADM-100 Practitioner Studio Administration · HDX-ADM-100 DX Security · HDX-ADM-100 DXClient*

1. The mental model: what is DX, administratively?

HCL Digital Experience (DX) is built on WebSphere Portal underneath. That matters because it explains *why* things are organized the way they are — everything (pages, content, apps, security) is a **resource in a portal object hierarchy**, and permissions cascade down that hierarchy like folder permissions on a filesystem.

Think of it like this:

DX	SAP Commerce / Hybris
WebSphere Portal core (page tree, resource hierarchy, security)	The platform/core layer underneath Accelerator
Practitioner Studio — the unified low-code admin/authoring UI	Backoffice — the unified admin cockpit
A DX “Sandbox” on HCL SoFy (Marketplace)	A cloud subscription environment (CCv2 / cloud sandbox), spun up via Helm/Kubernetes similar to how CCv2 spins up via its provisioning pipeline
Solution Console (sol-admin)	Cloud Portal / ops console for a CCv2 subscription

Key mindset shift: in Hybris, most configuration lives in **impex + Spring config + items.xml**, edited by developers and deployed via build. In DX, an enormous amount of “admin” work happens live, in-browser, in the Practitioner Studio — closer to doing everything through Backoffice’s Admin Cockpit rather than editing XML/Groovy files. DXClient and WebDAV exist precisely to give developers a code-first alternative to that browser-based workflow (see Course 2).

2. Practitioner Studio — your one console

Practitioner Studio is where administrators *and* developers *and* business users all work — just with different menu items visible depending on role. Its top-level sections:

- **Home** – dashboard/shortcuts
- **Administration** – global settings, security, site management (≈ Backoffice’s *System / Users / Type System* cockpits combined)
- **Web Content** – Content Composer/Authoring (≈ WCMS Cockpit)
- **Digital Assets** – DAM (≈ Media Cockpit)
- **Site Templates, Themes, Personalization** (≈ CMS Cockpit + Smart Merchandising rules, roughly)

About (under Administration → Settings) tells you the exact build/CF (Cumulative Fix) level — the DX equivalent of checking your Hybris patch/build number (`hybris-cli` or `system-check`), important because features and even lab steps differ by CF.

3. Security model — the biggest conceptual jump from Hybris

Hybris security is mostly **Type Permissions** (per item type × PrincipalGroup: create/read/change/remove) plus Spring Security roles for Backoffice widgets, and storefront auth is separate (customer groups, B2B user groups).

DX security is built around **Virtual Resources** and **Roles**, and it applies uniformly to *everything* — pages, portlets, content libraries, even system functions like “can a user self-register.” Think of a Virtual Resource as roughly analogous to a Hybris **item type** in the permission sense — it’s “a thing permissions attach to” — except in DX it also covers abstract system capabilities, not just data types.

DX concept	Closest Hybris analog
Virtual Resource (e.g. USER SELF ENROLLMENT, WCM REST SERVICE) Role (User / Editor / Manager / Administrator on a resource)	A Type-Permission-protected item type, or a protected Spring endpoint The CRUD bits in Type Permissions (Create/Read/Change/Delete), but expressed as named roles instead of checkboxes
Anonymous Portal User All Authenticated Portal Users dxAdministrators group	anonymous PrincipalGroup customergroup / logged-in PrincipalGroup admingroup / employee group with full Type Permissions
Resource Permissions → Assign Access → Edit Role → Add/Delete Allow Inheritance / Allow Propagation on a Web Content Library	Backoffice Type Permissions editor, PrincipalGroup × Type grid Type Permission inheritance from parent catalog/type, and permission propagation to sub-catalogs

The four concrete admin patterns from the lab, and their Hybris equivalent:

1. **Prevent self-registration** — remove Anonymous Portal User from the *Editor* role on the USER SELF ENROLLMENT virtual resource. → Hybris equivalent: disabling anonymous checkout/self-registration is usually a **business process / Spring config flag** (store.session.customer.previous-style flags, or simply not exposing a registration widget), not a Type Permission — DX treats it as a permission because *everything* in DX, even a UI capability, is a securable resource.
2. **Block anonymous access to a whole site** — remove Anonymous Portal User from the page/site's Set Page Permissions. → Hybris equivalent: a **CMS page/site restriction** (CMSPageRestriction requiring login), or removing the anonymous user group from a base store's session flow.
3. **Promote a user to admin** — add the user (by uid) to the dxAdministrators group. → Hybris equivalent: adding an Employee to the admingroup PrincipalGroup in Backoffice.
4. **Secure a Web Content Library** — set root-level access (Allow Inheritance/Propagation), then grant User role to specific groups on that library. → Hybris equivalent: setting **Type Permissions per Catalog** (read/write access differs per content catalog, e.g. marketing content vs. legal content), or Backoffice's item-level restriction views.

Practical rule of thumb carried over from Hybris instincts: just like in Hybris you always test permission changes by logging in as the actual restricted user (not just checking the config), the DX labs consistently tell you to **log out and re-test as the affected user** after every permission change — same discipline, same reason (cached sessions/permissions can lie to you).

4. DXClient — DX's "deployment CLI"

DXClient is a Node.js-based (or container-based) command-line tool used to **deploy and stage DX resources** (themes, web content libraries, Script Applications — covered in Course 2) from your local machine or CI pipeline to a DX server.

DXClient	Hybris equivalent
<pre>npm install -g @hcl-software/dxclient</pre>	Installing the ant/hybris-cli tooling that ships with the platform
<pre>dxclient -V (accept license, show version)</pre>	ant -version / checking the platform build
<pre>store/config.json (global connection settings: hostname, dxUsername, dxPassword, dxConnect creds)</pre>	local.properties / environment-specific config used by ant/impex import scripts
<pre>dxclient version-compat</pre>	A version-compatibility/health check — nothing exact in Hybris, closest is checking your extension's buildcallbacks.xml targets against the platform version
<pre>dxclient livesync pull-theme / push-theme</pre>	Roughly like pulling a Hybris Accelerator locally, editing, then redeploying — except DX does this as a live two-way sync against the running server rather than a full extension build
<pre>dxclient deploy-theme, deploy-scriptapplication</pre>	ant deploy / a CI step that packages and pushes an extension
<pre>DXConnect (must be configured on the DX server for DXClient to talk to it)</pre>	Roughly like enabling HAC remote access or an admin REST endpoint — a server-side prerequisite that must be turned on before the CLI tool can reach it

Important distinction from Hybris: DXClient doesn't compile/build Java — DX "extensions" in this developer sense are mostly **content-based artifacts** (themes = HTML/CSS/JS + WCM content; Script Applications = HTML/CSS/JS stored as a content item). So the "deployment" story is much closer to syncing static web assets + content JSON than to Hybris's Spring-bean/Java extension build-and-deploy cycle. You'll see this pattern much more in Course 2.

5. SoFy / HCLSoftware Marketplace — the sandbox layer

This isn't really "DX" itself — it's the **provisioning/hosting layer** you use to get a disposable DX server for learning, similar in spirit to how SAP gives you a CCv2 sandbox subscription or a local Hybris Docker image for a course.

- Deployed via **Helm** on Kubernetes ("DX Solution Modules" = a pre-bundled demo stack: DX + sample sites like Woodburn Stores/Studio/Healthcare, roughly analogous to Hybris's B2C/B2B Accelerator sample data + demo storefronts you get out of the box).
- **Solution Console** (sol-admin) gives you pod/resource/log visibility — this is your Kubernetes-dashboard-lite, the rough equivalent of watching CCv2 build/deploy logs or `kubectl get pods` if you've worked with a containerized Hybris deployment.
- Sandboxes have a TTL (session length) you can extend — purely a cost-control mechanic of the training environment, no direct product concept.

Summary cheat-table

If you know this in Hybris...	...think of this in DX
Backoffice	Practitioner Studio
Type Permissions grid	Virtual Resources + Roles (Resource Permissions)
PrincipalGroup (admingroup, customergroup)	User Group (dxAdministrators, Anonymous Portal User, All Authenticated Portal Users)
CMSRestriction on a page	Set Page Permissions on a site/page
ant deploy / CI pipeline	DXClient (deploy-theme, deploy-scriptapplication, livesync push/pull)
local.properties	DXClient store/config.json
CCv2 sandbox subscription	HCL SoFy sandbox (DX Solution Modules)
HAC / ops console	Solution Console (sol-admin)

-e

Course 1 — HDX-ADM-100: DX Administration for Beginners

PRACTICAL CHEAT SHEET

Use this as a quick-recall lab walkthrough — the “what did I click / type” reference, without needing to rewatch the videos.

A. Deploy a sandbox (HDX-INTRO)

1. Go to <https://marketplace.hcl-software.com> → **Resources** → **DX Marketplace**.
2. Filter: **Business Solutions** → **DX Solution Modules** → **LAUNCH IT** (~120 min startup, like waiting on a fresh CCv2 build).
3. Extend sandbox time if needed (default session ~6h).
4. Once ready: **Sandbox Links** → **Demo Home Page** → **Open** — gives you links to all demo sites + REST/GraphQL API explorers.
5. Optional: **Sandbox Admin Console** login → sol-admin / <password from console> — shows pod status, becomes “Ready” (green) once fully deployed → **General Information** / **Quick Links** for admin credentials.

Default creds used throughout every lab:

Purpose	User	Password
SoFy login	your email	your password
Solution Console	sol-admin	from solution console
DX admin	wpsadmin	wpsadmin
DX admin persona	hpappus	HCL-Dem0
DX developer persona	ghayes	HCL-Dem0

B. Practitioner Studio basics (HDX-ADM-100 Practitioner Studio Admin)

1. Log in as hpappus (or wpsadmin) → **Applications menu** → **Home**.
2. **Administration** → **Settings** = global settings; **About** = version/CF check.
3. **Applications menu** → **Administration** = top-level admin hub (Security, Site Management, Web Content, etc. all launch from here).

Create a new Web Content Library

1. **Applications menu** → **Web Content** → opens Content Composer.
 2. **Web Content Libraries** → **Create new library**.
 3. Name it, optionally set localization / “prohibit deletion” / “include default items.”
 4. Ensure it’s marked **available to developers**.
-

C. Security lab (HDX-ADM-100 DX Security)

1. Prevent self-registration

1. Log in as hpappus.
2. **Applications → Administration → Security → Resource Permissions → Virtual Resources.**
3. Find **USER SELF ENROLLMENT → Assign Access → Edit Role (Editor).**
4. Delete Anonymous Portal User → OK.
5. Back at the resource list → click **USER SELF ENROLLMENT → Apply → OK.**
6. **Test:** log out, go to the site's login page, click "Create an account" — option should be gone.

2. Block anonymous access to a site

1. **Administration → Site Management → Content Root.**
2. Find the site (e.g. Woodburn Studio) → **Set Page Permissions.**
3. **Edit Role (User)** → delete Anonymous Portal User → OK.
4. Back at site level → **Apply → OK.**
5. **Test:** log out — you should be redirected to the first site still accessible anonymously.

3. Promote a user to Administrator

1. **Administration → Security** (any Security option) → gives Users/Groups search.
2. Switch search type to **Users**, search by uid (e.g. ghayes) — optional: check **View membership** to confirm current groups.
3. Switch search type back to **User Groups**, find **dxAdministrators**, click **View membership.**
4. **Add Member** → search Users by uid → select → OK.
5. Confirm the target user now appears as a member.

4. Manage access on a Web Content Library

1. **Web Content → Web Content Libraries** → create/select a library.
2. Click the **key icon** (root access) → note **Allow Inheritance / Allow Propagation** toggles.
3. **Edit Role (User)** → **Add** → select All Authenticated Portal Users + Anonymous Portal User → OK.
4. Back out to **Resources → Apply → OK → Done.**

Pattern to memorize: **find resource → Edit Role → Add/Delete principals → back out → Apply → OK.** Every DX permission change follows this exact 5-step shape, whether it's a virtual resource, a page, or a library.

D. DXClient install & configure (HDX-ADM-100 DXClient)

Install (Node.js path — recommended)

```
node -v                                # confirm supported Node version first
npm install -g @hcl-software/dxclient
# use sudo if permission denied; omit -g for a local per-project install
```

First run & configure

```
cd ~/dxclient          # or any working directory
dxclient -V            # accept license on first run → prints version, e.g.
⇒ CF221.0.0
```

- First run auto-creates a store/ folder: store/logs/ and store/config.json.
- dxclient -h → list all commands.

Test connectivity (version-compat)

```
dxclient version-compat -h
dxclient version-compat -hostname dx.sbx0000.play.hclsofy.com \
  -dxConnectPort 443 -dxConnectUsername wpsadmin -dxConnectPassword wpsadmin
```

Find hostname / DXConnect creds under **SoFy → Sandbox Links → Configuration Wizard**.

Avoid repeating params every time — edit store/config.json

```
{
  "hostname": "dx.sbx0000.play.hclsofy.com",
  "dxConnectPort": 443,
  "dxConnectUsername": "wpsadmin",
  "dxConnectPassword": "wpsadmin"
}
```

Then just run: dxclient version-compat (no flags needed). Any CLI flag you *do* pass overrides the config file value.

Optional: container (Docker/Podman) path

```
# after downloading hcl-dxclient-image-*.zip and unzipping it:
docker load < dxclient.tar.gz
./bin/dxclient -V

# Podman alternative:
export CONTAINER_RUNTIME=podman
```

Mac M1/ARM issue → add --platform linux/amd64 to the \$CONTAINER_RUNTIME call inside bin/dxclient (around line 78).

Make DXClient runnable from any directory

```
pwd          # get current dxclient/bin path
export PATH=<working-directory>/bin:$PATH
echo $PATH   # verify
# persist on Mac/Linux:
echo 'export PATH=<working-directory>/dxclient/bin:$PATH' >> ~/.bash_profile
source ~/.bash_profile
```

Running `dxclient` in a *new* directory creates a **new** `store/config.json` there — copy your original config over to avoid reconfiguring every time (each working directory = its own local “workspace,” similar to how each Hybris extension has its own `local.properties` overrides).

Quick command reference (grows in Course 2)

Task	Command
Show help	<code>dxclient -h</code>
Show version	<code>dxclient -V</code>
Check server compatibility	<code>dxclient version-compat</code>
(Course 2) Pull theme locally	<code>dxclient livesync pull-theme</code>
(Course 2) Push theme to server	<code>dxclient livesync push-theme</code>
(Course 2) Deploy theme (CI-style)	<code>dxclient deploy-theme -themeName "<name>" -themePath <path></code>